Laboratorio de Java

Entrega final: sábado 7 de noviembre de 2026.

Trabajo en parejas; si la cantidad de estudiantes es impar, se habilitará un único equipo de tres integrantes.

Un repositorio de GitHub y un video explicativo por equipo. El video se alojará en otra plataforma y se enlazará desde el README.

Propósito y desafío

Transformar un prototipo genérico en una aplicación de escritorio funcional, utilizando Java y Swing. Cada equipo elegirá un problema concreto, fundamentará su propuesta y completará el sistema respetando la organización MVC, una fachada de acceso a las operaciones y la persistencia en JSON.

El laboratorio busca aplicar clases, objetos, encapsulamiento y paquetes, organizar responsabilidades, gestionar eventos de una interfaz y conservar información entre ejecuciones. Se evaluarán las decisiones implementadas y la capacidad de explicarlas.

Elección del proyecto

El sistema inicial será un gestor de registros con identificador, nombre, categoría y estado activo. Cada equipo definirá qué representan esos registros y adaptará sus atributos, categorías, reglas y operaciones al problema elegido.

Ejemplos posibles: recursos de un laboratorio, solicitudes de soporte, materiales de una biblioteca, actividades de un centro o inscripciones a talleres. Cambiar únicamente nombres, colores o textos no constituye una adaptación suficiente.

Punto de partida proporcionado

Se entregará un proyecto que compila, abre una ventana y muestra datos de ejemplo leídos desde un archivo JSON. La lectura y escritura del archivo estarán resueltas como infraestructura; el equipo deberá integrarlas con las operaciones que implemente.

Los botones de gestión mostrarán inicialmente un aviso de funcionalidad pendiente. Registrar, modificar y eliminar serán tareas del equipo. La base incluirá cinco tipos de componentes: JTextField, JComboBox, JCheckBox, JButton y JTable.

Todos los integrantes deberán participar en el desarrollo, conocer la solución completa y explicar sus aportes en el video. El reparto de tareas no exime de comprender el funcionamiento general.

URL: https://github.com/Domingo1987/prog2-lab2026

Requisitos funcionales

La aplicación final deberá permitir las siguientes acciones, con textos y comportamientos adaptados al problema elegido:

- Registrar elementos nuevos con un identificador que no se repita.
- Seleccionar un registro del listado, consultar sus datos y modificarlo sin crear duplicados.
- Eliminar un registro seleccionado, solicitando confirmación.
- Buscar o filtrar registros por al menos un criterio útil para el proyecto.
- Actualizar el listado después de cada operación y permitir limpiar el formulario.
- Guardar los cambios en JSON y recuperar la información al iniciar una nueva ejecución.

Componentes gráficos

Se deberán conservar o adaptar los cinco tipos de componentes incluidos y agregar al menos tres tipos diferentes que no estén en la base. El resultado tendrá, como mínimo, ocho tipos de componentes con un uso funcional.

Componente inicial	Función de referencia
JTextField	Ingresar el nombre o un dato textual breve.
JComboBox	Seleccionar una categoría.
JCheckBox	Indicar un estado lógico.
JButton	Ejecutar acciones del sistema.
JTable	Mostrar y seleccionar registros.

Para los componentes nuevos pueden elegir, por ejemplo, JTextArea para observaciones, JSpinner para cantidades, JRadioButton para opciones excluyentes, JList para una selección o JFileChooser para elegir un archivo. Pueden utilizar otros si justifican su función.

Se cuentan tipos de componentes, no cantidades de instancias: varios botones siguen contando como JButton. JLabel, JPanel, JFrame y los contenedores de apoyo no cuentan para alcanzar el mínimo. Los componentes nuevos deben afectar los datos o una acción real; no alcanza con mostrarlos.

Validación y mensajes

Implementar al menos tres validaciones pertinentes, incluyendo una regla propia del problema. Ejemplos: nombre obligatorio, cantidad positiva y prohibición de asignar un recurso fuera de servicio. Informar el motivo del error y permitir corregirlo sin cerrar la aplicación. Mostrar también confirmaciones cuando corresponda.

Arquitectura y responsabilidades

Mantener la separación del proyecto inicial. Se podrán agregar clases y subpaquetes cuando resulte necesario, sin concentrar toda la solución en la ventana principal.

Paquete o carpeta	Responsabilidad
app	Iniciar la aplicación y conectar sus objetos.
modelo	Representar las entidades y las reglas del problema.
vista	Presentar la interfaz Swing y los mensajes al usuario.
controlador	Recibir eventos y coordinar las acciones mediante la fachada.
fachada	Ofrecer operaciones simples y coordinar el modelo y la persistencia.
persistencia	Leer y escribir los registros en JSON.
datos	Contener el archivo JSON; no es un paquete Java.

La vista no debe leer ni escribir archivos ni decidir las reglas del negocio. El controlador utiliza la fachada para solicitar operaciones. La persistencia no debe depender de componentes gráficos. Las validaciones de negocio deben mantenerse en el modelo o en los servicios de lógica correspondientes.

Persistencia y manejo de errores

El archivo JSON debe incluir los datos necesarios para reconstruir los registros, también los atributos agregados por el equipo. No se requiere una base de datos ni conexión a Internet para utilizar la aplicación.

El sistema debe poder iniciar sin un archivo previo, comenzando con una colección vacía. Ante un archivo ilegible o un error de escritura, debe informar el problema sin cerrarse inesperadamente ni sobrescribir silenciosamente los datos que no pudo recuperar.

Comprobación del funcionamiento

Registrar en el README una tabla breve con la acción realizada, el resultado esperado y el resultado obtenido. Como mínimo, comprobar:

- Crear un registro válido y verificar que aparece en el listado.
- Intentar guardar datos que incumplan cada una de las tres validaciones.
- Modificar un registro y comprobar que conserva su identificador.
- Cancelar una eliminación y luego confirmarla con otro registro de prueba.
- Aplicar una búsqueda o filtro y comprobar los resultados.
- Crear y modificar datos, cerrar y volver a abrir para verificar su conservación.
- Iniciar sin archivo JSON y comprobar el manejo de un archivo inválido, usando copias de prueba.

Fundamentación y documentación

Incluir un archivo README.md en español, ubicado en la raíz del repositorio, con el nombre del proyecto, los nombres completos de los integrantes y estos apartados: problema y destinatarios; utilidad de la solución; adaptación del modelo y reglas; tres componentes nuevos y su función; organización de paquetes y recorrido de una operación; instrucciones de ejecución, versión de Java y dependencias; pruebas realizadas y limitaciones conocidas; aportes de cada integrante; enlace público al video explicativo.

Distinguir qué partes provienen del prototipo y cuáles fueron implementadas o modificadas por el equipo. Si se utilizan bibliotecas, tutoriales o herramientas de IA, indicar su uso. Todo código entregado debe poder explicarse.

Video explicativo

El video deberá alojarse en una plataforma externa, como Google Drive o YouTube, y enlazarse en una sección Video explicativo del README.md. No se deberá subir el archivo de video al repositorio.

El enlace deberá permitir ver el video sin iniciar sesión ni solicitar autorización. En Google Drive, configurar Cualquier persona con el enlace con permiso de lectura; en YouTube, utilizar un video público o no listado accesible mediante el enlace. Verificar el acceso desde una ventana privada o de incógnito antes de entregar.

Entregar un único video por equipo, de un máximo de cinco minutos, que sustituye la defensa oral. Debe mostrar la aplicación funcionando y fragmentos relevantes del código, con texto legible y audio

comprensible. No es necesario aparecer en cámara; todos los integrantes deben intervenir con su voz y explicar un aporte concreto.

El video debe presentar el problema y la solución, demostrar las funciones principales y una validación, identificar los tres componentes nuevos y explicar el recorrido de una operación por MVC, fachada y persistencia. Debe mostrar que los datos se conservan al cerrar y volver a abrir. No se espera leer todo el código ni explicar cada línea.

Entrega y organización del trabajo

La entrega final se realizará mediante un repositorio de GitHub por equipo, hasta el sábado 7 de noviembre de 2026. Se deberá entregar al docente la URL del repositorio y comprobar que puede acceder a su contenido. No se entregará un archivo ZIP.

El repositorio deberá utilizar el siguiente nombre estándar:

prog2-lab2026-[apellido1]_[apellido2]

Reemplazar los marcadores por los apellidos de los integrantes, en minúsculas y sin tildes, espacios ni corchetes. Ejemplo: prog2-lab2026-perez_gomez. En el único equipo de tres integrantes, agregar el tercer apellido: prog2-lab2026-perez_gomez_rodriguez.

El repositorio deberá incluir:

- Código fuente completo, organizado en los paquetes previstos.
- Archivo JSON de ejemplo, sin datos personales reales.
- Dependencias o configuración necesaria para compilar y ejecutar la aplicación.
- Archivo README.md en la raíz, con la fundamentación, las instrucciones de ejecución, las pruebas, los aportes de cada integrante y el enlace público al video alojado en otra plataforma.

La aplicación deberá poder ejecutarse siguiendo las instrucciones del README. Antes de entregar, comprobar que la versión final esté subida a GitHub y que el enlace al video funcione sin permisos adicionales.

Fecha	Avance recomendado
Sábado 17 de octubre	Definir el problema, revisar la base y planificar los componentes.
Sábado 31 de octubre	Tener la gestión, los componentes y el guardado integrados.
Sábado 7 de noviembre	Entregar la URL del repositorio final, con el proyecto probado y el enlace público al video en el README.

Evaluación

Criterio	Puntos	Se valorará
Funcionamiento y datos	35	Gestión, búsqueda, validaciones, JSON y manejo de errores.
Arquitectura y código	25	MVC, fachada, responsabilidades claras y código comprensible.
Interfaz gráfica	20	Cinco tipos iniciales más tres nuevos, útiles e integrados.
Fundamentación y comunicación	20	README, pruebas, decisiones justificadas y video de hasta cinco minutos con participación de todos.

Total: 100 puntos.

Se valorará lo que efectivamente funcione y pueda verificarse. Incorporar componentes sin una finalidad o entregar operaciones pendientes no acredita su implementación.